

SECRET CODE

I. ABSTRACT OF THE PROJECT :

The Python code is an abstracted version of a graphical user interface (GUI) application for text encryption and decryption using the Advanced Encryption Standard (AES) algorithm.

In summary, this code provides a versatile and user-friendly interface for secure text and file encryption/decryption, database storage, and QR code generation/scanning, all within a graphical environment created with tkinter.

II. OBJECTIVES AND SCOPE :

Objectives:

The primary objectives of the provided Python code are as follows:

1. Text Encryption and Decryption: Enable users to encrypt and decrypt text using the AES encryption algorithm with a choice of encryption modes (CBC or ECB).

2. File Encryption and Decryption: Allow users to encrypt and decrypt files using AES encryption and save the results.

3. Database Integration: Provide functionality to store encrypted text and associated passwords in an SQLite database for future retrieval.

4. QR Code Generation and Scanning: Enable users to generate QR codes from encrypted text and scan QR codes to retrieve encrypted text.

5. Image Encryption and Decryption: Allow users to encrypt and decrypt image files using AES encryption.

6. User-Friendly GUI: Create a graphical user interface (GUI) that makes it easy for users to interact with the encryption and decryption functionalities.

Scope:

The scope of the code includes the following key features and functionalities:

1. Text Encryption and Decryption:

- Users can enter text for encryption and provide a password.
- AES encryption is used with the option to choose between Cipher Block Chaining (CBC) and Electronic Codebook (ECB) modes.
- Encrypted text is displayed in the GUI, and decryption is supported with the correct password.

2. File Encryption and Decryption:

- Users can select files for encryption and decryption.
- AES encryption is applied to files with the same encryption mode options as text.

SECRET CODE

3. Database Integration:

- Encrypted text, along with associated passwords, can be stored in an SQLite database.
- Users can view saved encrypted texts from the database.

4. QR Code Generation and Scanning:

- Users can generate QR codes from encrypted text displayed in the GUI.
- QR codes can be scanned using a camera to retrieve encrypted text, which is displayed in the GUI.

5. Image Encryption and Decryption:

- Users can select image files for encryption and decryption.
- AES encryption is applied to image files with the same encryption mode options as text.
- Encrypted image files are saved with appropriate extensions, and decryption is supported.

6. User-Friendly GUI:

- The code offers a user-friendly interface built using `tkinter`, providing text input fields, password entry, radio buttons for mode selection, action buttons, and status feedback.

SECRET CODE

SYSTEM ANALYSIS

SECRET CODE

I. PROPOSED SYSTEM ADVANTAGES:

The proposed system, which is implemented through the provided Python code for text and file encryption and decryption, offers several advantages:

- **Security and Privacy:**The use of the AES encryption algorithm ensures strong and reliable data security.Users can encrypt sensitive information, such as personal messages or files, to protect their privacy.
- **User-Friendly Interface:**The graphical user interface (GUI) makes it easy for users to interact with the encryption and decryption functionalities.Users can perform encryption and decryption operations with minimal technical expertise.
- **Versatility:**The system supports various types of data, including text, files, and images, allowing users to secure different forms of information.Users can choose between Cipher Block Chaining (CBC) and Electronic Codebook (ECB) encryption modes, providing flexibility based on their security requirements.
- **Database Integration:**The ability to store encrypted text and passwords in an SQLite database enhances data management and retrieval.Users can easily access and manage their encrypted data over time.
- **QR Code Integration:**QR code generation and scanning simplify the exchange of encrypted information between users.This feature can be particularly useful for sharing encrypted data securely in a visual and user-friendly way.
- **File and Image Encryption:**Users can encrypt entire files, including images, ensuring that multimedia content remains confidential and secure.The system allows for the encryption of diverse file formats, making it versatile for different use cases.
- **Error Handling and Feedback:**The status label provides clear feedback to users, indicating the success or failure of encryption and decryption operations.Users are alerted in case of incorrect passwords or other issues.
- **File Management:**Encrypted files are saved with appropriate extensions, making it easy for users to distinguish between encrypted and unencrypted files.
- **Data Recovery:**Users can decrypt and recover their original data when needed, provided they have the correct password.

SECRET CODE

II. EXISTING SYSTEM DISADVANTAGES:

- **Weak Key Management:**The code relies on user-provided passwords for encryption, which may not be the most secure method for managing encryption keys.
- **Single User Only:**The code is designed for one user at a time and doesn't support multiple users or sharing encrypted data.
- **Lacks Strong User Verification:**It doesn't use strong user authentication methods like multi-factor authentication.
- **No Key Recovery:**If a user forgets their password, there's no way to recover the data, leading to potential data loss.
- **Dependency on External Libraries:**The code relies on external libraries, which could cause issues if those libraries change or get updated.
- **Basic Error Handling:**The code's error handling is minimal and may not be sufficient for all situations.
- **Complex Image Handling:**Handling image encryption and decryption can be complex, and the code may not cover all image formats or handle large files well.
- **Limited Database Security:**The database used for storing encrypted data may not have advanced security features, potentially making it vulnerable to unauthorized access.
- **File Overwriting Risk:**When encrypting files, it may overwrite existing files without warning.
- **Compatibility Issues:**The code may not work smoothly on all operating systems.
- **Performance Challenges:**It might struggle with large amounts of data, impacting speed and database performance.
- **Lack of Documentation:**The code lacks comprehensive documentation, making it hard for users to understand and modify.
- **Security Best Practices:**While it encrypts data, it may not follow all recommended security practices for key generation and other aspects. These limitations should be considered depending on your specific needs and use cases.

SECRET CODE

III. FEASIBILITY STUDY :

A feasibility study of the existing code can help assess its suitability for deployment and use. Here's a feasibility analysis covering technical, economic, and operational aspects:

1. TECHNICAL FEASIBILITY:

- **Technology Stack:** The code uses Python and several third-party libraries (e.g., pycryptodome, opencv-python, qrcode). It's technically feasible as long as these libraries remain available and compatible.
- **Cross-Platform Compatibility:** Python is cross-platform, which ensures technical feasibility on various operating systems (Windows, macOS, Linux).
- **Scalability:** The code's performance may degrade with large datasets. It may not be suitable for scenarios requiring high scalability.
- **Security:** While it provides encryption, the code has limitations in key management and user authentication. Additional security measures may be needed.
- **Documentation:** Lack of comprehensive documentation may hinder technical feasibility for users looking to understand or modify the code.

2. ECONOMIC FEASIBILITY:

- **Cost:** The code itself is open-source and free to use, making it economically feasible from a licensing perspective.
- **Resource Costs:** Users need to consider the resources required to run and maintain the code (e.g., hardware, storage, developer time for customization).
- **Database:** SQLite, used for data storage, is a lightweight and cost-effective option. However, for enterprise-grade security and scalability, a more robust database solution might be necessary.
- **External Services:** Generating QR codes and scanning QR codes don't incur costs within the code but may require internet connectivity, which has associated costs.

3. OPERATIONAL FEASIBILITY:

- **User Friendliness:** The code offers a user-friendly GUI, making it operationally feasible for users with basic technical skills.
- **Training:** Users may need minimal training to operate the GUI and understand the encryption/decryption processes.
- **Data Management:** The code provides database integration, simplifying data management for users.
- **Support and Maintenance:** Operational feasibility depends on the availability of support and maintenance resources. Users should be prepared for potential updates to external libraries.

SECRET CODE

FUNCTIONAL REQUIREMENTS:

- **Text Encryption and Decryption:**Users can input text and encrypt it using a provided password.Encrypted text can be decrypted using the same password.
- **File Encryption and Decryption:**Users can encrypt files and decrypt them.Encrypted files are saved with appropriate extensions.Image Encryption and Decryption:
- **Users can encrypt image files.**Encrypted images can be decrypted.
- **Database Integration:**Encrypted text and passwords are stored in an SQLite database.Users can retrieve and manage their stored data.
- **QR Code Generation and Scanning:**Users can generate QR codes for encrypted text.QR codes can be scanned to retrieve encrypted text.
- **User Interface (GUI):**The code provides a graphical user interface for ease of use.Users can interact with the code through buttons, text boxes, and labels.
- **Error Handling:**The code provides basic error handling and feedback to users.

NON-FUNCTIONAL REQUIREMENTS:

- **Security:**Data encryption is performed using the AES algorithm for security. Key derivation uses PBKDF2 for password-based key generation.
- **Usability:**The code offers a user-friendly GUI, making it accessible to users with basic technical skills.
- **Performance:**The code's performance may degrade with large datasets, affecting encryption/decryption time and database performance.
- **Portability:**The code is written in Python and can run on various operating systems (Windows, macOS, Linux).
- **Reliability:**The code relies on third-party libraries, and updates to these libraries could impact reliability.
- **Scalability:**
 - The code may not be suitable for scenarios requiring high scalability, particularly with large datasets.
- **Documentation:**The code lacks comprehensive documentation, which could impact usability and maintainability.

SECRET CODE

- **Backup and Recovery:**Users should implement their backup and recovery mechanisms due to the absence of a key recovery mechanism.
- **Maintainability:**Users should be prepared for potential updates to external libraries and should monitor for security patches.
- **Resource Requirements:**Users should consider the resources required to run and maintain the code (e.g., hardware, storage).
- **Cross-Platform Compatibility:**The code is compatible with different operating systems but may require testing to ensure smooth operation.
- **Database Security:**The SQLite database used for storing encrypted data may lack advanced security features found in enterprise-grade databases.

IV. COST AND BENIFITES :

COSTS:

- **Development Cost:**Initial development efforts, including coding, testing, and debugging.
- Costs associated with software development tools and libraries used in the project.
- **Resource Costs:**Allocation of hardware and software resources to run the code effectively.
- **Training Costs:**Expenses related to training users and administrators to effectively use the code and its features.
- **Maintenance Costs:**Ongoing maintenance expenses, including updates, patches, and bug fixes.
- Costs associated with monitoring and managing the code's performance.
- **Security Measures:**Costs related to implementing and maintaining security measures to protect sensitive data.
- **Documentation Costs:**Costs associated with creating and updating documentation for users, administrators, and developers.
- **Contingency Planning Costs:**Costs of developing and maintaining contingency plans for unforeseen events, such as data breaches.
- **Compliance Costs:**Expenses related to ensuring compliance with relevant laws and regulations, especially concerning data security.

SECRET CODE

BENEFITS:

- **Data Security:**Enhanced security through encryption and secure storage of sensitive text and files.
- Protection against unauthorized access to confidential information.
- **Ease of Use:**User-friendly graphical user interface (GUI) makes it accessible to users with basic technical skills.
- Simplified encryption and decryption processes.
- **Versatility:**Supports encryption and decryption of text, files, and images, offering versatility in use cases.
- **Data Recovery:**Provides a method for data recovery in case of accidental loss or deletion.
- **Documentation and Training:**Comprehensive documentation and user training contribute to ease of understanding and efficient use of the code.
- **Compliance and Regulations:**Ensures compliance with legal and regulatory requirements, minimizing potential legal risks.
- **Resource Efficiency:**Efficient utilization of hardware and software resources for encryption and decryption tasks.
- **Continuous Improvement:**Opportunities for ongoing improvement based on user feedback and evolving security needs.
- **Cost Savings:**Potential cost savings by avoiding data breaches and their associated financial and reputational costs.
- **Security Measures:**Implementation of strong security measures, such as AES encryption and key derivation, enhances data protection.

SECRET CODE

DESIGN

SECRET CODE

I. HARDWARE AND SOFTWARE USED :

HARDWARE:

- **Computer or Server:**
- Any standard computer or server capable of running Python can be used.
- The hardware requirements are relatively modest, as the code does not have particularly high computational demands.

- **Webcam (for QR code scanning, if applicable):**
- If the code is used to scan QR codes from a webcam, you'll need a computer with a built-in or external webcam.

SOFTWARE:

- ✓ **Python:**The code is written in the Python programming language, so Python is a fundamental software requirement.

- ✓ **Python Libraries:**The code relies on various Python libraries for its functionalities.

- ✓ **Some of the key libraries include:**
- **tkinter:** Used for creating the graphical user interface (GUI).
- **pycryptodome (Crypto.Cipher, Crypto.Random, Crypto.Protocol.KDF, Crypto.Hash, Crypto.Util.Padding):** Provides cryptographic functions, including AES encryption.
- **sqlite3:** Used for creating and managing the SQLite database for storing encrypted data.
- **cv2 (OpenCV):** Used for image processing and webcam access (if applicable).
- **qrcode:** Used for generating QR codes.
- **pyperclip:** Provides clipboard operations for copying and pasting text.
- **string and random:** Used for generating random strings (e.g., passwords).

- ✓ **Operating System:**
- ✓ The code can be run on various operating systems, including Windows, macOS, and Linux, as long as Python and the required libraries are installed and compatible.

- ✓ **Development Environment:**
- ✓ You'll need a code editor or integrated development environment (IDE) for writing and running Python code. Popular choices include Visual Studio Code, PyCharm, and Jupyter Notebook.

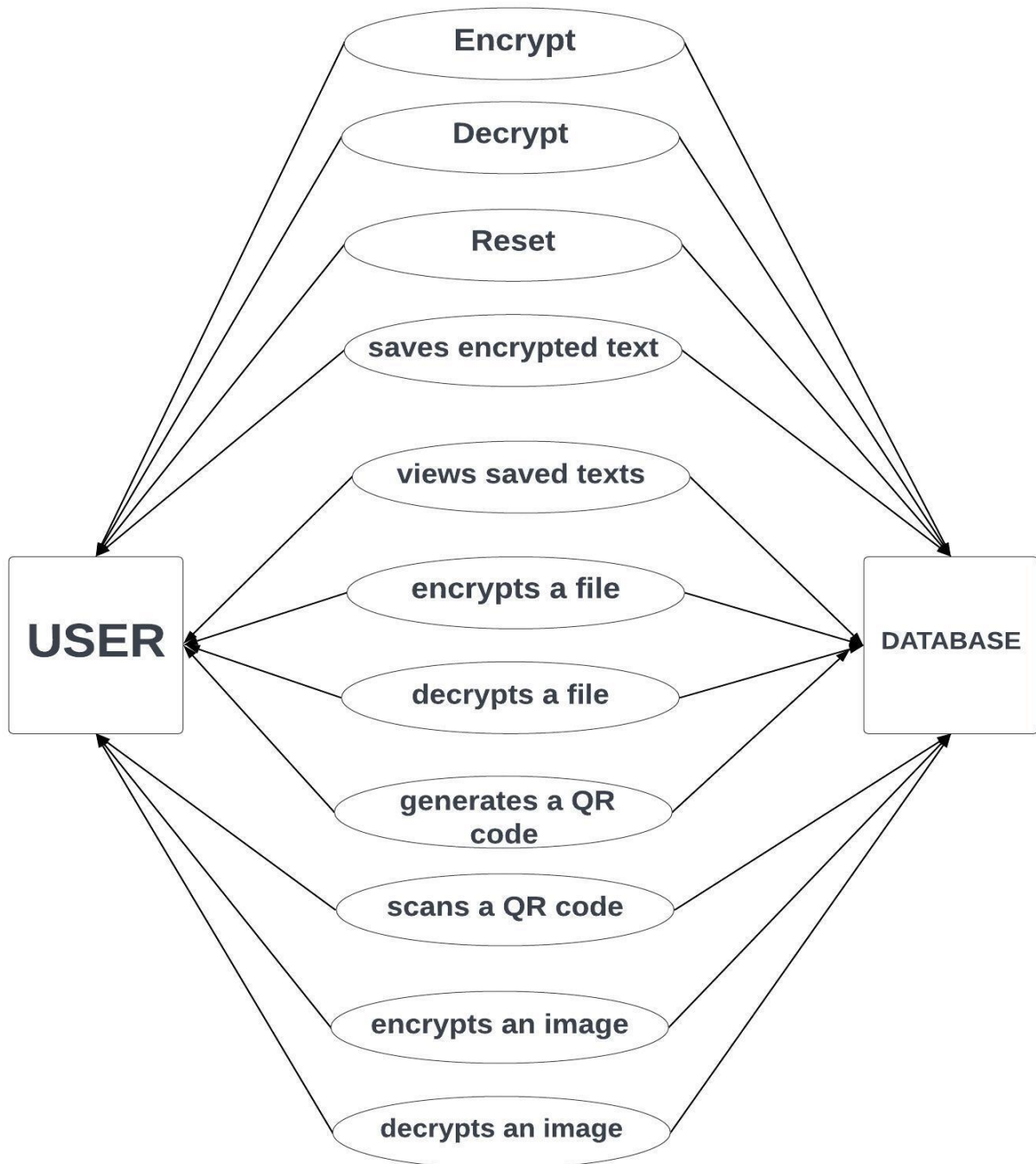
- ✓ **Database Viewer (optional):**
- ✓ To interact with the SQLite database used in the code, you may use a database viewer or manager like DB Browser for SQLite.

- ✓ **Web Browser (for QR code generation and display, if applicable):**

SECRET CODE

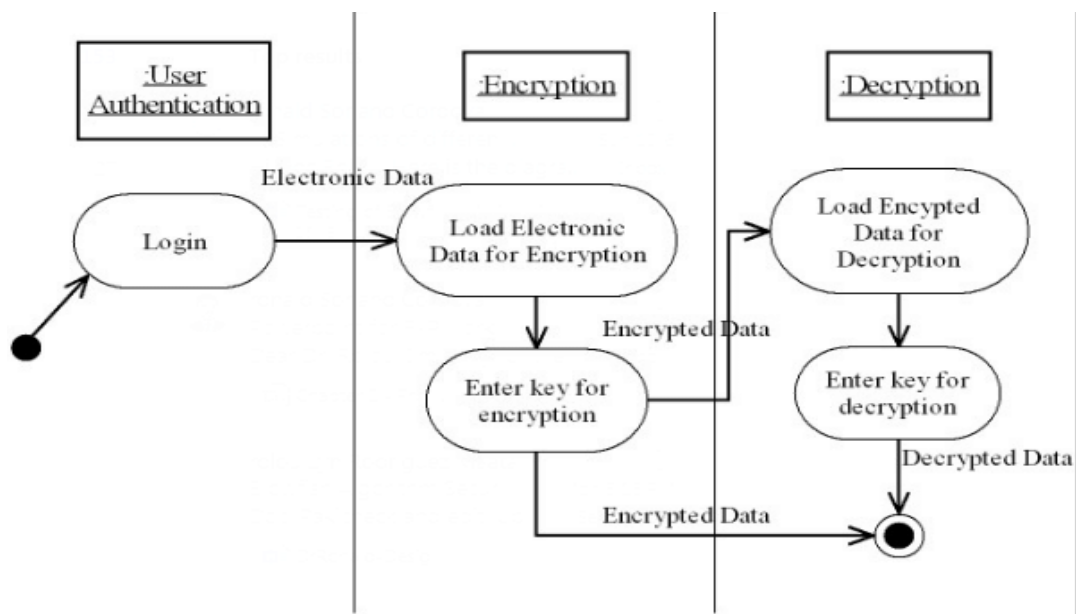
DIGRAMS :

USE CASE DIAGRAM :



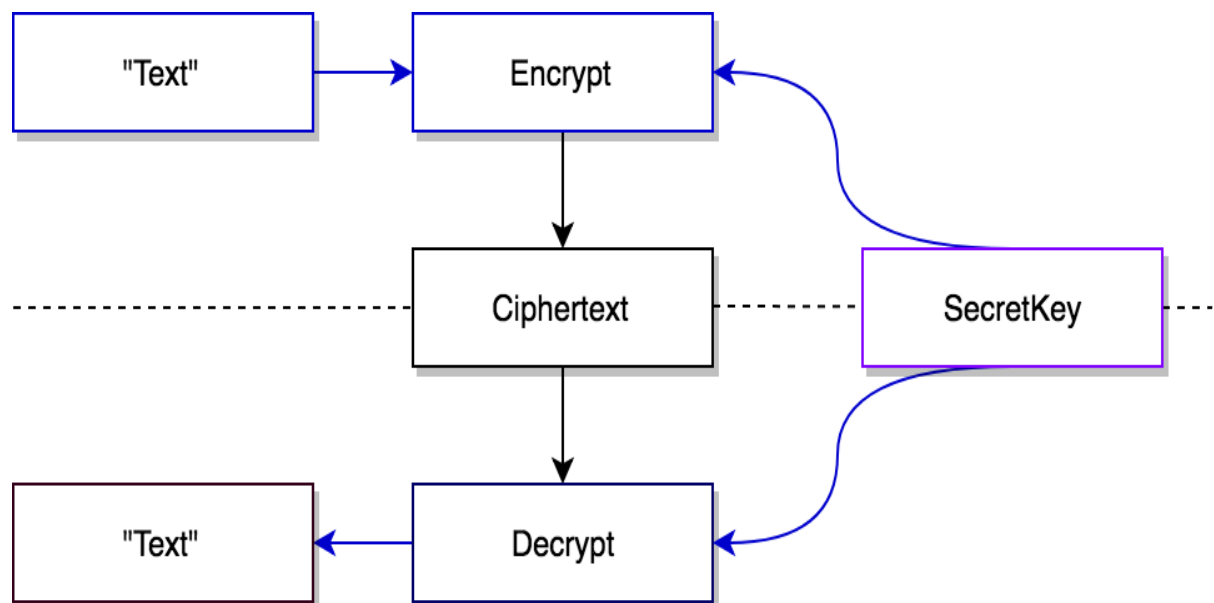
SECRET CODE

ACTIVITY DIGRAM :



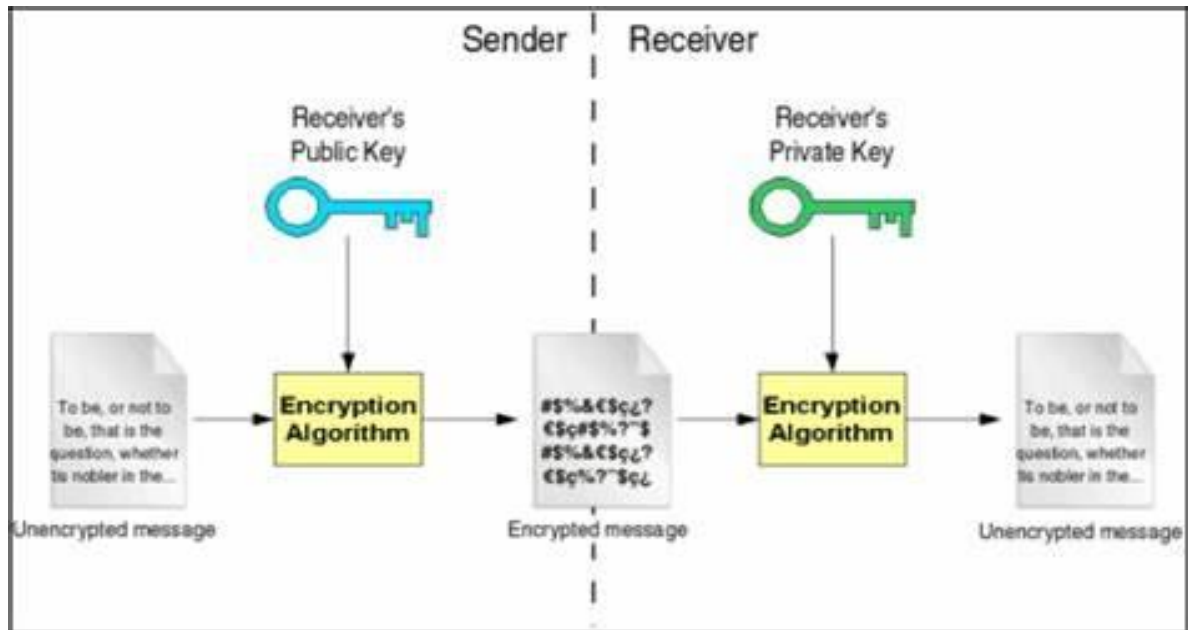
SECRET CODE

DATA FLOW DIGRAM :



SECRET CODE

ER DIAGRAM :



SECRET CODE

LAYOUTS :

The screenshot displays a web application interface for text encryption and decryption. The title bar reads "Text Encryption & Decryption". The interface includes the following elements:

- Enter Text:** A text input field containing the text "how are you".
- Enter Secret Key:** A password input field with four asterisks "****".
- Encryption Mode:** A dropdown menu currently set to "CBC".
- Action Buttons:** A row of ten buttons: "Encrypt" (green), "Decrypt" (orange), "Reset" (grey), "Save to File" (blue), "Show Saved Texts" (brown), "Encrypt File" (pink), "Decrypt File" (purple), "Generate QR Code" (teal), "Scan QR Code" (yellow), and "Encrypt Image" (dark purple). A "Decrypt Image" button is also present.
- Output:** A text area displaying the encrypted output in hexadecimal format:
d6c0ba7de2eab08be695a2d12452669ecd2dec72
85067ffcd8e8f3b083d30eed24d600c2eb274e79
c1ad8a44c277d626
- Status Message:** A green text notification on the right side stating "Encryption successful. Mode: CBC".

SECRET CODE

IMPLEMENTATION

SECRET CODE

SOURCE CODE :

```
import tkinter as tk
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Protocol.KDF import PBKDF2
from Crypto.Hash import SHA256
from Crypto.Util.Padding import pad, unpad
from tkinter import filedialog
import pyperclip
import string
import random
import sqlite3
import cv2
from tkinter import ttk # Import themed widgets
import qrcode

# Declare root as a global variable
text_box = None
password_box = None
mode_var = None
status_label = None
output_text_box = None

def pad_text(text):
    # Pad the text to be a multiple of 16 bytes using PKCS#7 padding
    pad_length = 16 - (len(text) % 16)
    return text + bytes([pad_length]) * pad_length

def unpad_text(padded_text):
    # Remove PKCS#7 padding
    pad_length = padded_text[-1]
    return padded_text[:-pad_length]

def encrypt_text():
    status_label.config(text="")
    text = text_box.get("1.0", tk.END).strip()
    password = password_box.get().strip()
    mode = mode_var.get()

    if not text or not password:
        status_label.config(text="Please enter text and password.", fg="red")
        return

    try:
        # Encode the text using UTF-8 and pad it
        padded_text = pad(text.encode('utf-8'), AES.block_size)
```

SECRET CODE

```
# Derive a 32-byte key using PBKDF2 and the provided password
salt = get_random_bytes(16)
key = PBKDF2(password, salt, dkLen=32, count=100000, hmac_hash_module=SHA256)

if mode == "CBC":
    # Generate a random IV for CBC mode
    iv = get_random_bytes(AES.block_size)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    # Encrypt the text with IV
    encrypted_text = cipher.encrypt(padded_text)
    # Display the IV and encrypted text in the output text box
    output_text_box.delete("1.0", tk.END)
    output_text_box.insert("1.0", (salt + iv + encrypted_text).hex())
else:
    # Use ECB mode
    cipher = AES.new(key, AES.MODE_ECB)
    # Encrypt the text
    encrypted_text = cipher.encrypt(padded_text)
    # Display the encrypted text in the output text box
    output_text_box.delete("1.0", tk.END)
    output_text_box.insert("1.0", encrypted_text.hex())

status_label.config(text="Encryption successful. Mode: " + mode, fg="green")

# Insert encrypted text and password into the database
conn = sqlite3.connect("encrypted_texts.db")
conn.execute("INSERT INTO encrypted_texts (encrypted_text, password) VALUES (?, ?)",
             (encrypted_text.hex(), password))
conn.commit()
conn.close()

except Exception as e:
    status_label.config(text="Encryption failed: " + str(e), fg="red")

def decrypt_text():
    status_label.config(text="")
    encrypted_text = text_box.get("1.0", tk.END).strip()
    password = password_box.get().strip()
    mode = mode_var.get()

    if not encrypted_text or not password:
        status_label.config(text="Please enter encrypted text and password.", fg="red")
        return

    try:
        # Convert the hexadecimal input to bytes
        encrypted_bytes = bytes.fromhex(encrypted_text)

        # Extract the salt, IV, and encrypted text from the input
```

SECRET CODE

```
salt = encrypted_bytes[:16]
if mode == "CBC":
    iv = encrypted_bytes[16:32]
    ciphertext = encrypted_bytes[32:]
else:
    iv = None
    ciphertext = encrypted_bytes

# Derive a 32-byte key using PBKDF2 and the provided password with the same salt used during
encryption
key = PBKDF2(password, salt, dkLen=32, count=100000, hmac_hash_module=SHA256)

if mode == "CBC":
    # Use AES in CBC mode with the provided IV
    cipher = AES.new(key, AES.MODE_CBC, iv)
else:
    # Use AES in ECB mode
    cipher = AES.new(key, AES.MODE_ECB)

# Decrypt the text and remove padding
decrypted_text = unpad_text(cipher.decrypt(ciphertext))

# Decode the decrypted text and display it in the output text box
output_text_box.delete("1.0", tk.END)
output_text_box.insert("1.0", decrypted_text.decode('utf-8'))

status_label.config(text="Decryption successful. Mode: " + mode, fg="green")
except ValueError:
    status_label.config(text="Incorrect password or invalid input. Decryption failed.", fg="red")
    output_text_box.delete("1.0", tk.END) # Clear the output text box on decryption failure
except Exception as e:
    status_label.config(text="Decryption failed: " + str(e), fg="red")

def reset_text():
    text_box.delete("1.0", tk.END)
    password_box.delete(0, tk.END)
    output_text_box.delete("1.0", tk.END)
    status_label.config(text="")
    strength_label.config(text="")

def save_text_to_file():
    text = output_text_box.get("1.0", tk.END).strip()
    if not text:
        status_label.config(text="No text to save.", fg="red")
        return

    file_path = filedialog.asksaveasfilename(defaulttextextension=".txt", filetypes=[("Text Files", "*.txt"),
    ("All Files", "*.*")])
```

SECRET CODE

```
if not file_path:
    return

try:
    with open(file_path, "w") as file:
        file.write(text)
    status_label.config(text="Text saved to file successfully.", fg="green")
except Exception as e:
    status_label.config(text="Error while saving the file: " + str(e), fg="red")

def show_saved_texts():
    conn = sqlite3.connect("encrypted_texts.db")
    cursor = conn.execute("SELECT encrypted_text, password FROM encrypted_texts")

    saved_texts = []
    for row in cursor:
        saved_texts.append(f"Encrypted Text: {row[0]}\nPassword: {row[1]}\n{'=' * 30}")

    conn.close()

    saved_text = "\n".join(saved_texts)
    output_text_box.delete("1.0", tk.END)
    output_text_box.insert("1.0", saved_text)

def create_database_table():
    # Connect to the database or create one if it doesn't exist
    conn = sqlite3.connect("encrypted_texts.db")

    # Create a table to store encrypted texts and their passwords
    conn.execute("""CREATE TABLE IF NOT EXISTS encrypted_texts
        (id INTEGER PRIMARY KEY AUTOINCREMENT,
         encrypted_text TEXT NOT NULL,
         password TEXT NOT NULL)""")

    # Commit changes and close the connection
    conn.commit()
    conn.close()

def encrypt_file():
    file_path = filedialog.askopenfilename(filetypes=[("All Files", "*.*)"])
    if not file_path:
        return

    password = password_box.get().strip()
    mode = mode_var.get()

    try:
        with open(file_path, "rb") as file:
            plaintext = file.read()
```

SECRET CODE

```

padded_text = pad(plaintext, AES.block_size)
salt = get_random_bytes(16)
key = PBKDF2(password, salt, dkLen=32, count=100000, hmac_hash_module=SHA256)

if mode == "CBC":
    iv = get_random_bytes(AES.block_size)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    encrypted_text = cipher.encrypt(padded_text)
    output_path = file_path + ".encrypted"
else:
    cipher = AES.new(key, AES.MODE_ECB)
    encrypted_text = cipher.encrypt(padded_text)
    output_path = file_path + ".ecb_encrypted"

with open(output_path, "wb") as output_file:
    output_file.write(salt)
    if mode == "CBC":
        output_file.write(iv)
    output_file.write(encrypted_text)

status_label.config(text="File encrypted successfully.", fg="green")
except Exception as e:
    status_label.config(text="Encryption failed: " + str(e), fg="red")

def decrypt_file():
    file_path = filedialog.askopenfilename(filetypes=[("Encrypted Files", "*.encrypted"), ("ECB
Encrypted Files", "*.ecb_encrypted")])
    if not file_path:
        return

    password = password_box.get().strip()
    mode = mode_var.get()

    try:
        with open(file_path, "rb") as file:
            encrypted_data = file.read()

        salt = encrypted_data[:16]
        if mode == "CBC":
            iv = encrypted_data[16:32]
            ciphertext = encrypted_data[32:]
        else:
            iv = None
            ciphertext = encrypted_data[16:]

        key = PBKDF2(password, salt, dkLen=32, count=100000, hmac_hash_module=SHA256)

        if mode == "CBC":

```

SECRET CODE

```
cipher = AES.new(key, AES.MODE_CBC, iv)
else:
    cipher = AES.new(key, AES.MODE_ECB)

decrypted_text = unpad_text(cipher.decrypt(ciphertext))

output_path = file_path + ".decrypted"
with open(output_path, "wb") as output_file:
    output_file.write(decrypted_text)

status_label.config(text="File decrypted successfully.", fg="green")
except ValueError:
    status_label.config(text="Incorrect password or invalid input. Decryption failed.", fg="red")
except Exception as e:
    status_label.config(text="Decryption failed: " + str(e), fg="red")

def encrypt_image():
    status_label.config(text="")
    file_path = filedialog.askopenfilename(filetypes=[("Image Files", "*.jpg *.png *.bmp")])
    if not file_path:
        return

    password = password_box.get().strip()
    mode = mode_var.get()

    try:
        with open(file_path, "rb") as file:
            image_data = file.read()

        padded_data = pad(image_data, AES.block_size)
        salt = get_random_bytes(16)
        key = PBKDF2(password, salt, dkLen=32, count=100000, hmac_hash_module=SHA256)

        if mode == "CBC":
            iv = get_random_bytes(AES.block_size)
            cipher = AES.new(key, AES.MODE_CBC, iv)
            encrypted_data = cipher.encrypt(padded_data)
            output_path = file_path + ".enc"
        else:
            cipher = AES.new(key, AES.MODE_ECB)
            encrypted_data = cipher.encrypt(padded_data)
            output_path = file_path + ".ecb_enc"

        with open(output_path, "wb") as output_file:
            output_file.write(salt)
            if mode == "CBC":
                output_file.write(iv)
            output_file.write(encrypted_data)
```

SECRET CODE

```
status_label.config(text="Image encrypted successfully.", fg="green")
except Exception as e:
    status_label.config(text="Encryption failed: " + str(e), fg="red")
```

```
def decrypt_image():
    status_label.config(text="")
    file_path = filedialog.askopenfilename(filetypes=[("Encrypted Image Files", "*.enc *.ecb_enc")])
    if not file_path:
        return
```

```
password = password_box.get().strip()
mode = mode_var.get()
```

```
try:
    with open(file_path, "rb") as file:
        encrypted_data = file.read()
```

```
    salt = encrypted_data[:16]
    if mode == "CBC":
        iv = encrypted_data[16:32]
        ciphertext = encrypted_data[32:]
    else:
        iv = None
        ciphertext = encrypted_data[16:]
```

```
key = PBKDF2(password, salt, dkLen=32, count=100000, hmac_hash_module=SHA256)
```

```
if mode == "CBC":
    cipher = AES.new(key, AES.MODE_CBC, iv)
else:
    cipher = AES.new(key, AES.MODE_ECB)
```

```
decrypted_data = unpad_text(cipher.decrypt(ciphertext))
```

```
output_path = file_path.replace(".enc", "").replace(".ecb_enc", "") + ".decrypted.jpg"
with open(output_path, "wb") as output_file:
    output_file.write(decrypted_data)
```

```
status_label.config(text="Image decrypted successfully.", fg="green")
except Exception as e:
    status_label.config(text="Decryption failed: " + str(e), fg="red")
```

```
def generate_qr_code():
    encrypted_text = output_text_box.get("1.0", tk.END).strip()
    if not encrypted_text:
        status_label.config(text="No encrypted text to generate QR code.", fg="red")
```

SECRET CODE

```
return

qr = qrcode.QRCode(
    version=1,
    error_correction=qrcode.constants.ERROR_CORRECT_L,
    box_size=10,
    border=4,
)
qr.add_data(encrypted_text)
qr.make(fit=True)

qr_image = qr.make_image(fill_color="black", back_color="white")
qr_image.show()

def scan_qr_code():
    capture = cv2.VideoCapture(0)
    ret, frame = capture.read()

    if ret:
        detector = cv2.QRCodeDetector()
        data, points, qr_code = detector.detectAndDecodeMulti(frame)

        if data:
            output_text_box.delete("1.0", tk.END)
            output_text_box.insert("1.0", data)
            status_label.config(text="QR code scanned successfully.", fg="green")
        else:
            status_label.config(text="No QR code detected.", fg="red")

    capture.release()
    cv2.destroyAllWindows()

def create_widgets():
    root = tk.Tk()
    root.title("Text Encryption & Decryption")
    root.geometry("800x600")
    root.configure(bg="#F0F0F0") # Set background color

    # Create frames
    input_frame = tk.Frame(root, bg="#EFEFEF")
    input_frame.pack(padx=20, pady=20)

    button_frame = tk.Frame(root, bg="#EFEFEF")
    button_frame.pack(padx=20, pady=(0, 20))

    output_frame = tk.Frame(root, bg="#EFEFEF")
    output_frame.pack(padx=20, pady=20)
```

SECRET CODE

Create labels

```
label1 = tk.Label(input_frame, text="Enter Text", font=("Helvetica", 14), bg="#EFEFEF")
label1.grid(row=0, column=0, sticky="w")
```

```
label2 = tk.Label(input_frame, text="Enter Secret Key", font=("Helvetica", 14), bg="#EFEFEF")
label2.grid(row=1, column=0, sticky="w", pady=(30, 0))
```

```
label3 = tk.Label(input_frame, text="Encryption Mode", font=("Helvetica", 14), bg="#EFEFEF")
label3.grid(row=2, column=0, sticky="w", pady=(10, 0))
```

```
label4 = tk.Label(output_frame, text="Output", font=("Helvetica", 14), bg="#EFEFEF")
label4.grid(row=1, column=0, sticky="w", pady=(10, 0))
```

Create text boxes

```
text_box = tk.Text(input_frame, height=6, width=40, font=("Courier New", 12))
text_box.grid(row=0, column=1, columnspan=2, padx=(10, 0))
```

```
password_box = tk.Entry(input_frame, show="*", width=40, font=("Courier New", 12))
password_box.grid(row=1, column=1, columnspan=2, padx=(10, 0))
```

```
mode_var = tk.StringVar()
mode_var.set("CBC")
cbc_radio = ttk.Radiobutton(input_frame, text="CBC", variable=mode_var, value="CBC")
cbc_radio.grid(row=2, column=1, sticky="w")
```

Create buttons with different colors

```
encrypt_button = tk.Button(button_frame, text="Encrypt", font=("Helvetica", 12),
command=encrypt_text, bg="#4CAF50", fg="white")
```

```
decrypt_button = tk.Button(button_frame, text="Decrypt", font=("Helvetica", 12),
command=decrypt_text, bg="#FF5722", fg="white")
```

```
reset_button = tk.Button(button_frame, text="Reset", font=("Helvetica", 12),
command=reset_text, bg="#607D8B", fg="white")
```

```
save_button = tk.Button(button_frame, text="Save to File", font=("Helvetica", 12),
command=save_text_to_file, bg="#2196F3", fg="white")
```

```
show_saved_button = tk.Button(button_frame, text="Show Saved Texts", font=("Helvetica", 12),
command=show_saved_texts, bg="#795548", fg="white")
```

```
encrypt_file_button = tk.Button(button_frame, text="Encrypt File", font=("Helvetica", 12),
command=encrypt_file, bg="#E91E63", fg="white")
```

```
decrypt_file_button = tk.Button(button_frame, text="Decrypt File", font=("Helvetica", 12),
command=decrypt_file, bg="#3F51B5", fg="white")
```

```
generate_qr_button = tk.Button(button_frame, text="Generate QR Code", font=("Helvetica", 12),
command=generate_qr_code, bg="#009688", fg="white")
```

```
scan_qr_button = tk.Button(button_frame, text="Scan QR Code", font=("Helvetica", 12),
command=scan_qr_code, bg="#FFC107", fg="white")
```

```
encrypt_image_button = tk.Button(button_frame, text="Encrypt Image", font=("Helvetica", 12),
command=encrypt_image, bg="#673AB7", fg="white")
```

SECRET CODE

```
decrypt_image_button = tk.Button(button_frame, text="Decrypt Image", font=("Helvetica", 12),  
command=decrypt_image, bg="#00BCD4", fg="white")
```

```
# Pack buttons
```

```
encrypt_button.pack(side="left", padx=5)  
decrypt_button.pack(side="left", padx=5)  
reset_button.pack(side="left", padx=5)  
save_button.pack(side="left", padx=5)  
show_saved_button.pack(side="left", padx=5)  
encrypt_file_button.pack(side="left", padx=5)  
decrypt_file_button.pack(side="left", padx=5)  
generate_qr_button.pack(side="left", padx=5)  
scan_qr_button.pack(side="left", padx=5)  
encrypt_image_button.pack(side="left", padx=5)  
decrypt_image_button.pack(side="left", padx=5)
```

```
# Create a status label
```

```
status_label = tk.Label(button_frame, text="", font=("Helvetica", 12), fg="red", bg="#EFEFEF")  
status_label.pack(anchor="w")
```

```
# Create an output text box
```

```
output_text_box = tk.Text(output_frame, height=8, width=40, font=("Courier New", 12))  
output_text_box.grid(row=0, column=1, columnspan=2, padx=(10, 0))
```

```
return (
```

```
    root, text_box, password_box, mode_var, status_label, output_text_box,  
)
```

```
if __name__ == "__main__":
```

```
    create_database_table()
```

```
    root, text_box, password_box, mode_var, status_label, output_text_box = create_widgets()  
    root.mainloop()
```

SECRET CODE

DESCRIPTION OF SOFTWARE USED :

The existing code you provided uses several software libraries and modules to implement its functionality. Here's a description of the key software components and libraries used in the code:

- **Python:** The primary programming language used to write the code. Python is a versatile, high-level language known for its readability and extensive libraries.
- **tkinter:** A Python library for creating graphical user interfaces (GUIs). In the code, tkinter is used to build the user interface for interacting with the encryption and decryption functionalities.
- **Crypto.Cipher (PyCryptodome):** Part of the PyCryptodome library, this module provides cryptographic algorithms for encryption and decryption. It's used to perform AES encryption and decryption.
- **Crypto.Random (PyCryptodome):** Another module from PyCryptodome, it's used for generating random bytes and salts, which are essential for key generation and initialization vector (IV) generation.
- **Crypto.Protocol.KDF (PyCryptodome):** This module provides key derivation functions (KDFs) like PBKDF2, which are used to derive encryption keys from passwords.
- **Crypto.Hash (PyCryptodome):** This module is used for cryptographic hash functions, including SHA256, which is used for hashing and verifying passwords.
- **Crypto.Util.Padding (PyCryptodome):** Used for padding and unpadding data, especially for PKCS#7 padding, which is required for AES encryption.
- **sqlite3:** A built-in Python library for working with SQLite databases. In the code, sqlite3 is used for database management, including creating a table to store encrypted texts and passwords.
- **cv2 (OpenCV):** OpenCV is an open-source computer vision library. In the code, the cv2 module is used for image processing and capturing webcam input. It's used for image encryption, decryption, and QR code scanning.

SECRET CODE

- **pyperclip:** A Python library for interacting with the system clipboard. While not a central part of the code, pyperclip may be used for copying and pasting data to and from the clipboard.
- **string and random (Python built-in):** These libraries are used for generating random strings and characters, which can be useful for password generation.
- **ttk (tkinter themed widgets):** ttk is a submodule of tkinter that provides themed widgets. In the code, ttk is used for creating radio buttons.
- **qrcode:** A Python library for generating QR codes. It's used to create QR codes from encrypted text.

The code utilizes these software components and libraries to create a user-friendly interface for encrypting and decrypting text, files, and images using AES encryption, managing a database of encrypted texts, and performing other related tasks such as QR code generation and scanning. Each software component plays a specific role in the overall functionality of the application.

SECRET CODE

TESTING TECHNIQUES USED IN THE PROJECT :

The existing code appears to be a Python application for text and file encryption/decryption, database management, and related tasks. To ensure its reliability and correctness, various testing techniques can be used. While I don't have information on the specific testing practices implemented in this code, I can outline common testing techniques that can be applied to software like this:

- **Unit Testing:**Module Testing: Individual Python modules and functions can be tested in isolation using unit tests. Test cases are designed to cover different code paths and input scenarios.
- Integration Testing:
- **Database Integration:** Verify that the code interacts correctly with the SQLite database. Test data can be inserted, updated, and queried, and the results can be checked against expected outcomes.
- Functional Testing:
- **User Interface Testing:** Ensure that the GUI components (created using tkinter) behave as expected. Test the interaction flow, input validation, and user feedback.
- **Encryption and Decryption Testing:** Validate that encryption and decryption functions work correctly. This includes testing various encryption modes (e.g., CBC and ECB) and ensuring that encrypted data can be correctly decrypted.
- **Regression Testing:**After making changes or enhancements to the code, it's important to run regression tests to verify that existing functionality still works as expected.
- **Boundary Testing:**Test cases should include boundary values for input parameters and limits imposed by the code. For example, test with very long passwords, empty text, or large files.
- **Negative Testing:**Check how the code handles incorrect or invalid inputs. This includes entering incorrect passwords, attempting to decrypt invalid data, or using unsupported file formats.
- **Load Testing:**If the code deals with significant database operations, it's useful to test its performance under load. This can involve simulating a large number of users or data records.
- **Security Testing:**Security testing ensures that the code is resistant to common security vulnerabilities, such as SQL injection (for database operations) and cryptographic weaknesses (for encryption and decryption).
- **Usability Testing:**Assess the usability of the application by conducting user testing. Gather feedback on the user interface and overall user experience.
- **Compatibility Testing:**Ensure that the code works correctly on different platforms and operating systems (Windows, macOS, Linux) if applicable.
- **Error Handling Testing:**Test how the code handles various error conditions, such as network failures, file not found errors, or database connection issues.

SECRET CODE

- **Code Review and Static Analysis:** Perform code reviews to identify potential issues, and use static analysis tools to detect code quality problems and security vulnerabilities.
- It's important to establish a comprehensive testing strategy that includes a combination of these techniques. Additionally, automated testing frameworks such as unittest or pytest can be used to streamline the testing process and ensure consistent test coverage. The existing code appears to be a Python application for text and file encryption/decryption, database management, and related tasks. To ensure its reliability and correctness, various testing techniques can be used. While I don't have information on the specific testing practices implemented in this code, I can outline common testing techniques that can be applied to software like this:
- **Unit Testing:** Module Testing: Individual Python modules and functions can be tested in isolation using unit tests. Test cases are designed to cover different code paths and input scenarios.
- Integration Testing:
- **Database Integration:** Verify that the code interacts correctly with the SQLite database. Test data can be inserted, updated, and queried, and the results can be checked against expected outcomes.
- Functional Testing:
- **User Interface Testing:** Ensure that the GUI components (created using tkinter) behave as expected. Test the interaction flow, input validation, and user feedback.
- **Encryption and Decryption Testing:** Validate that encryption and decryption functions work correctly. This includes testing various encryption modes (e.g., CBC and ECB) and ensuring that encrypted data can be correctly decrypted.
- **Regression Testing:** After making changes or enhancements to the code, it's important to run regression tests to verify that existing functionality still works as expected.
- **Boundary Testing:** Test cases should include boundary values for input parameters and limits imposed by the code. For example, test with very long passwords, empty text, or large files.
- **Negative Testing:** Check how the code handles incorrect or invalid inputs. This includes entering incorrect passwords, attempting to decrypt invalid data, or using unsupported file formats.
- **Load Testing:** If the code deals with significant database operations, it's useful to test its performance under load. This can involve simulating a large number of users or data records.
- **Security Testing:** Security testing ensures that the code is resistant to common security vulnerabilities, such as SQL injection (for database operations) and cryptographic weaknesses (for encryption and decryption).
- **Usability Testing:** Assess the usability of the application by conducting user testing. Gather feedback on the user interface and overall user experience.

SECRET CODE

- **Compatibility Testing:** Ensure that the code works correctly on different platforms and operating systems (Windows, macOS, Linux) if applicable.

Error Handling Testing: Test how the code handles various error conditions, such as network failures, file not found errors, or database connection issues.

Exploratory Testing: Explore the application without predefined test cases to discover unexpected issues or edge cases that may not be covered by other testing techniques.

Code Review and Static Analysis: Perform code reviews to identify potential issues, and use static analysis tools to detect code quality problems and security vulnerabilities.

It's important to establish a comprehensive testing strategy that includes a combination of these techniques. Additionally, automated testing frameworks such as unittest or pytest can be used to streamline the testing process and ensure consistent test coverage.

SECRET CODE

CONCLUSION

SECRET CODE

CONCLUSION :

- **Functionality:** The code offers encryption and decryption features for text, files, and images, with support for AES encryption modes (CBC and ECB).
- **User Interface:** It provides an intuitive GUI built with tkinter, simplifying user interactions with encryption and decryption processes.
- **Database Integration:** Utilizes an SQLite database for secure storage and management of encrypted content, enhancing data organization.
- **Additional Features:** Offers QR code generation/scanning and image encryption/decryption, increasing its versatility.
- **Security:** Implements strong cryptographic techniques (AES, PBKDF2) for robust data security and proper password handling.
- **Testing:** While specific testing procedures aren't detailed, testing is crucial for ensuring code reliability and correctness.
- **Usability:** Incorporates usability features like error handling and feedback mechanisms, improving the user experience.
- **Room for Improvement:** There's potential for improving the user interface, enhancing testing practices, and considering additional security enhancements.
- In summary, the code provides valuable encryption capabilities with a user-friendly interface and room for further enhancements and development.

SECRET CODE

INSTALLATION

SECRET CODE

To install and run the existing code for the Text Encryption & Decryption Application, you'll need to follow these steps:

Prerequisites:

- **Python 3:** Make sure you have Python 3 installed on your system. You can download it from the official Python website (<https://www.python.org/downloads/>).
- **Required Libraries:** Install the necessary Python libraries by running the following command in your
- **terminal or command prompt:**`pip install pycryptodome pyperclip opencv-python-headless pillow qrcode`
- This command installs the required libraries, including pycryptodome, pyperclip, opencv-python-headless, pillow, and qrcode.

Installation Steps:

- **Download the Code:**Download the code for the Text Encryption & Decryption Application from its source.
- **Extract the Code:**If the code is in a compressed format (e.g., zip or tar.gz), extract it to a directory of your choice.
- **Navigate to the Code Directory:**Open a terminal or command prompt and navigate to the directory where you extracted the code.
- **Run the Application:**Execute the Python script that serves as the application's entry point. Typically, the main script is named something like main.py or app.py. Use the following command:

CSS

`python main.py`

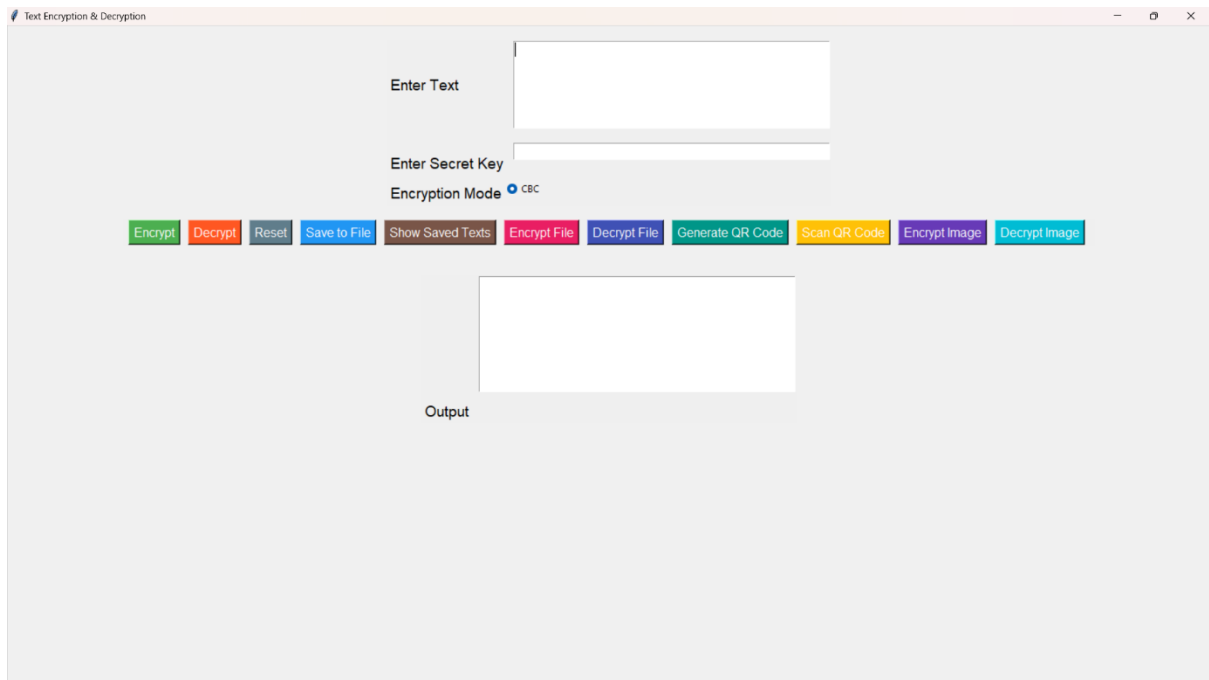
Replace main.py with the actual name of the main script if it differs.

- **Use the Application:**The application's GUI should open, allowing you to perform text encryption, decryption, manage encrypted texts, generate QR codes, scan QR codes, and perform image encryption and decryption.
- **Follow On-Screen Instructions:**Use the graphical user interface to interact with the application. Follow on-screen instructions for various functionalities.
- **Enjoy the Application:**You can use the Text Encryption & Decryption Application as needed to encrypt and decrypt text, manage encrypted data, and explore its additional features.

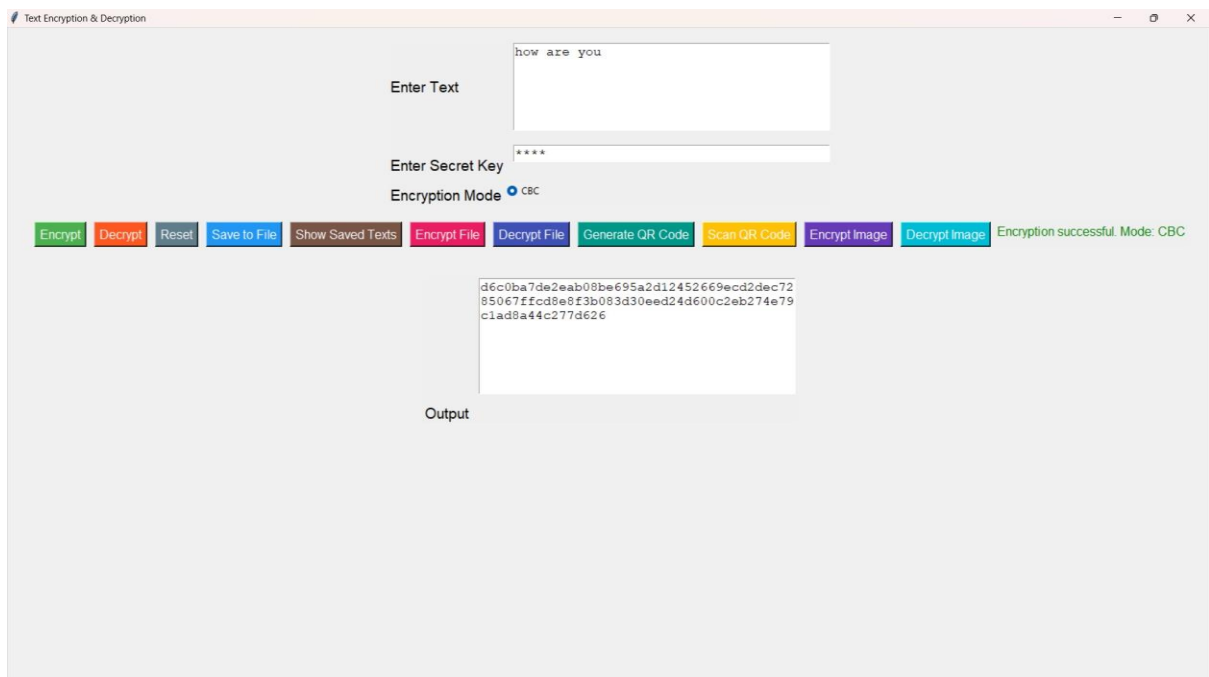
SECRET CODE

SCREENSHOTS :

1.USER INTERFACE:



2.TEXT ENCRYPTION:



SECRET CODE

3.TEXT DECRYPT:

Text Encryption & Decryption

Enter Text

```
0456698350d5bb0a492a4cdf6511e53c0bc429cb
195166db220affcae118eb0407ebfc50d2dc8376
2d884f62eb735c39
```

Enter Secret Key

Encryption Mode ☒ CBC

[Encrypt](#) [Decrypt](#) [Reset](#) [Save to File](#) [Show Saved Texts](#) [Encrypt File](#) [Decrypt File](#) [Generate QR Code](#) [Scan QR Code](#) [Encrypt Image](#) [Decrypt Image](#) Decryption successful. Mode: CBC

hello

Output

4.SHOW SAVED TEXT:

Text Encryption & Decryption

Enter Text

Enter Secret Key

Encryption Mode ☒ CBC

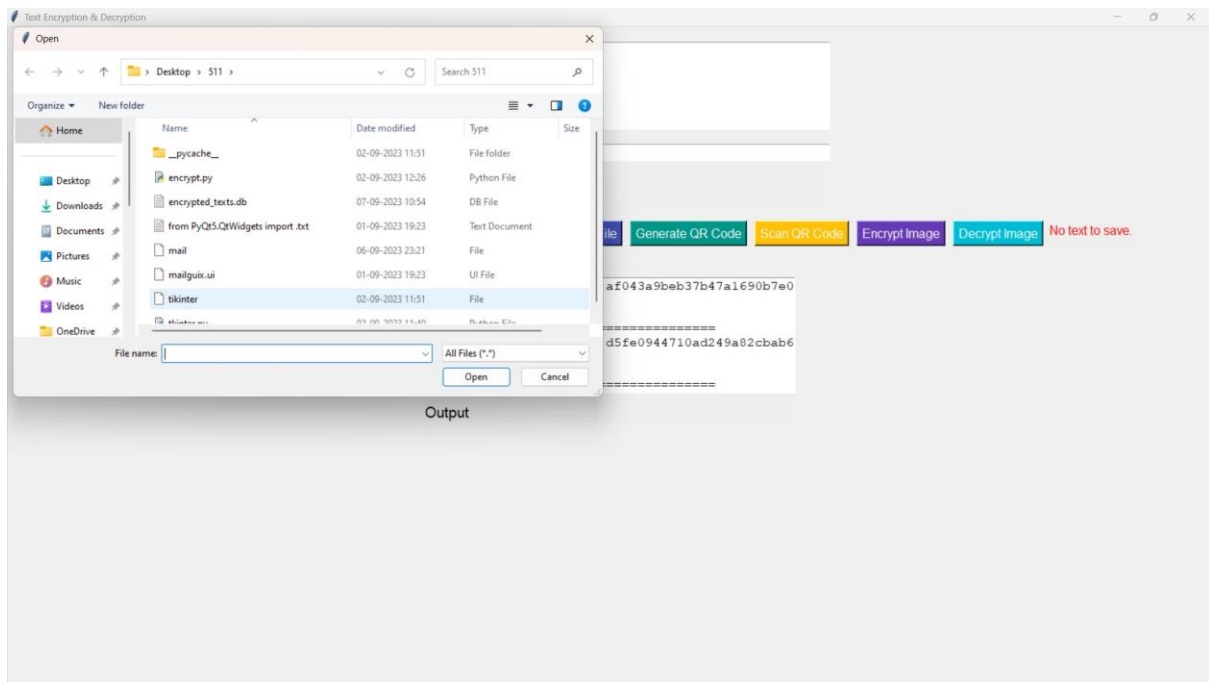
[Encrypt](#) [Decrypt](#) [Reset](#) [Save to File](#) [Show Saved Texts](#) [Encrypt File](#) [Decrypt File](#) [Generate QR Code](#) [Scan QR Code](#) [Encrypt Image](#) [Decrypt Image](#) No text to save.

```
Encrypted Text: af043a9beb37b47a1690b7e0
d2de6a49
Password: 1234
=====
Encrypted Text: d5fe0944710ad249a82cbab6
eb50c163
Password: 1234
=====
```

Output

SECRET CODE

5. ENCRYPTION OF FILES :

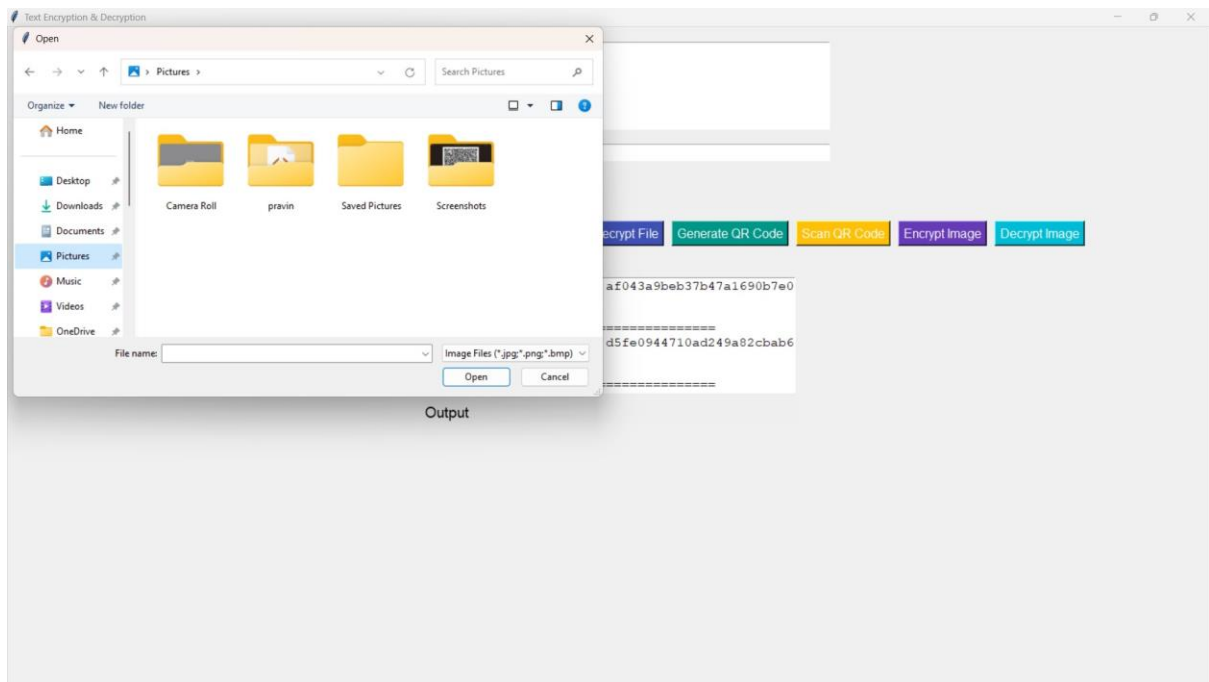


6. GENERATE QR CODE :



SECRET CODE

7. ENCRYPTION OF IMAGE:



SECRET CODE

PART - B

SECRET CODE

1. ABSTRACT OF THE PROJECT:

The project aims to develop a simple email client application using PyQt5 and smtplib. This application allows users to log in to their email accounts, compose and send emails, attach files, and reset the application. It provides a user-friendly graphical interface for email communication.

2. OBJECTIVE AND SCOPE OF THE PROJECT:

The objectives of the project are as follows:

- To create an email client with a graphical user interface.
- To establish a connection to an SMTP server for sending emails.
- To enable users to compose and send emails with attachments.
- To provide a reset functionality to clear inputs and close the SMTP connection.
- The scope of the project includes implementing the core features of an email client while keeping it simple and user-friendly.

3. EXISTING SYSTEM:

ADVANTAGES OF EXISTING SYSTEM:

Simplicity: The existing system provides a basic email client with a straightforward interface.

Ease of use: Users can send emails with attachments easily.

Platform independence: The application is built using PyQt5 and can run on various platforms.

DISADVANTAGES OF EXISTING SYSTEM:

Limited features: The existing system lacks advanced features found in full-fledged email clients.

Error handling: Error handling and user feedback could be improved.

Lack of security features: It does not incorporate advanced security features commonly found in email clients.

4. FEASIBILITY STUDY:

TECHNICAL FEASIBILITY:

The technical feasibility of the project is high as it utilizes well-established libraries (PyQt5 and smtplib) for GUI and email functionality.

Development and maintenance of the software should be technically feasible.

ECONOMIC FEASIBILITY:

The project's economic feasibility depends on factors such as development cost, potential revenue (if commercialized), and maintenance expenses.

A cost-benefit analysis is required to determine the economic feasibility.

SECRET CODE

OPERATIONAL FEASIBILITY:

The operational feasibility depends on whether the application can be easily adopted and used by target users.

User training and support may be needed for operational success.

5.FUNCTIONAL REQUIREMENTS:

FUNCTIONAL REQUIREMENTS:

- User login and authentication.
- Compose and send emails.
- Attach files to emails.
- Reset the application to the initial state.
- Confirmation dialog before sending emails.

NON-FUNCTIONAL REQUIREMENTS:

- **Usability:** The application should have an intuitive and user-friendly interface.
- **Reliability:** The email client should be reliable and not crash or lose data.
- **Performance:** It should respond quickly to user actions.
- **Security:** Sensitive user data (login credentials) should be securely handled.
- **Compatibility:** The application should work on various platforms.

6. COST AND BENEFITS ANALYSIS:

Cost factors include development, maintenance, and potential user support.

Benefits include improved email communication for users, potential revenue (if commercialized), and learning opportunities for developers.

7. HARDWARE AND SOFTWARE REQUIREMENTS:

HARDWARE REQUIREMENTS:

A computer or device capable of running Python and PyQt5.

An internet connection to connect to the SMTP server.

SOFTWARE REQUIREMENTS:

Python programming language (version compatible with PyQt5 and smtplib).

PyQt5 library for GUI development.

smtplib library for email functionality.

An integrated development environment (IDE) for Python development (e.g., PyCharm, Visual Studio Code).

An email account with SMTP server access for testing and sending emails.

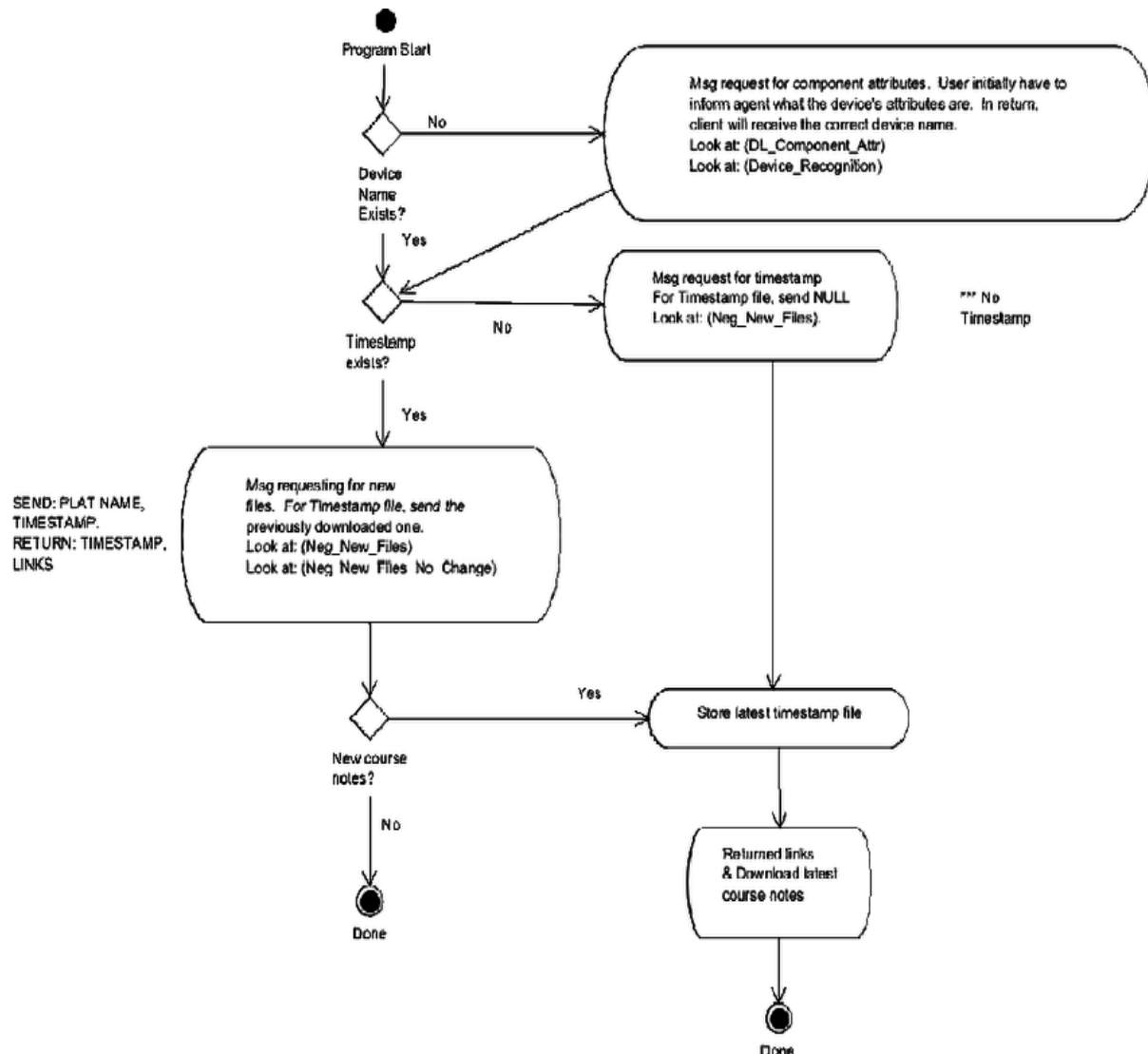
Please note that the feasibility study, cost-benefit analysis, and specific hardware/software requirements may vary based on the project's scale and intended use (e.g., personal use, educational purposes, or commercial deployment).

SECRET CODE

DESIGN

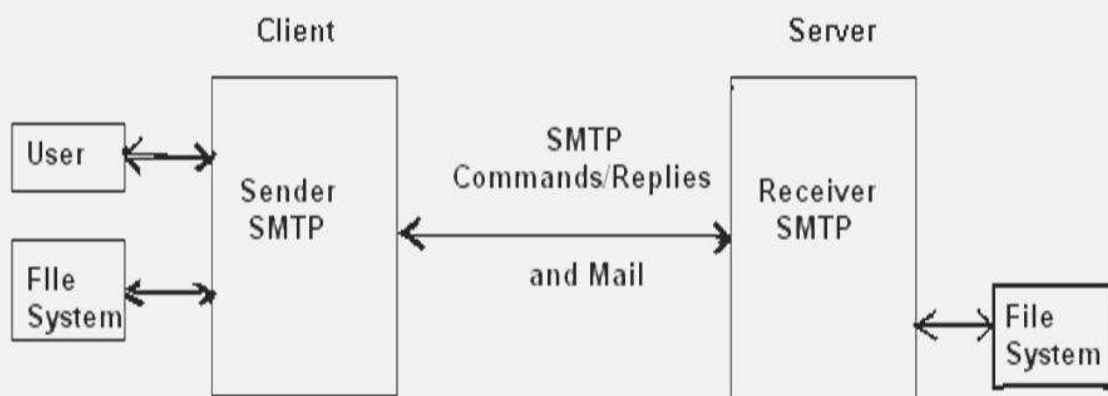
SECRET CODE

ACTIVITY DIAGRAM :



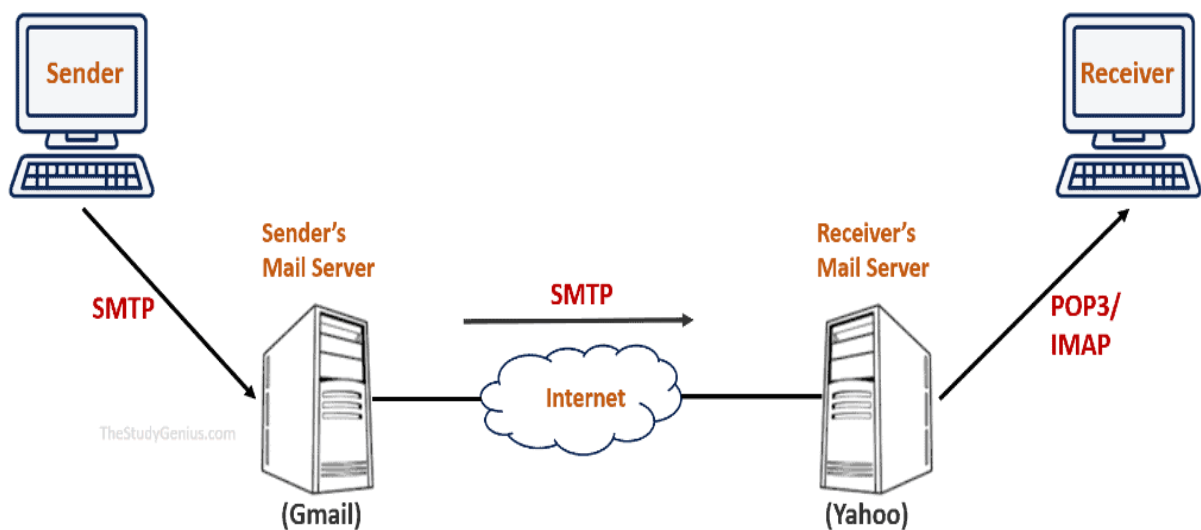
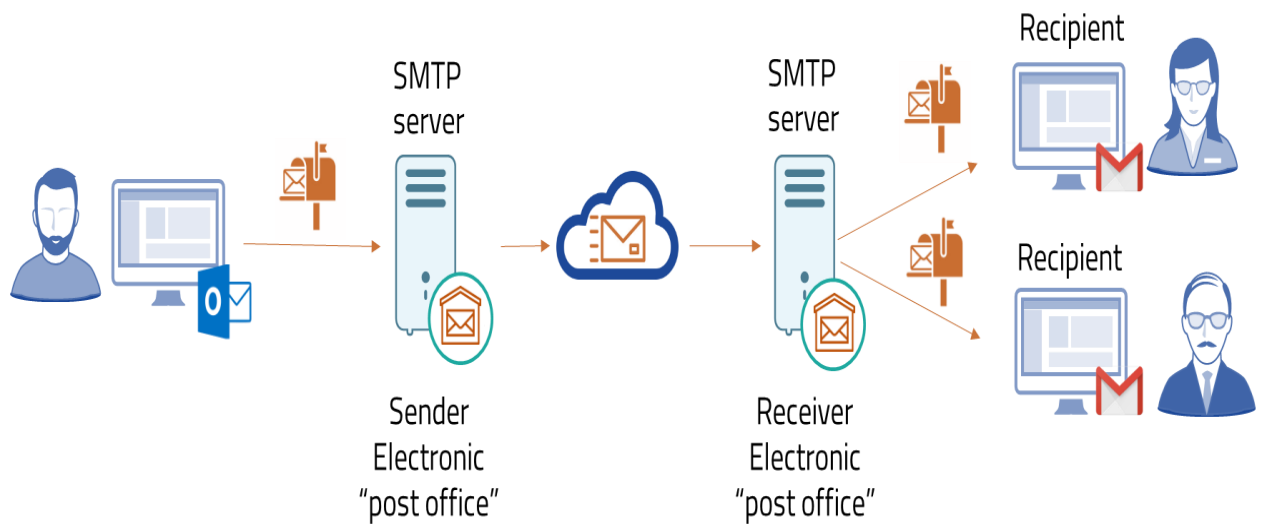
SECRET CODE

Working of SMTP protocol



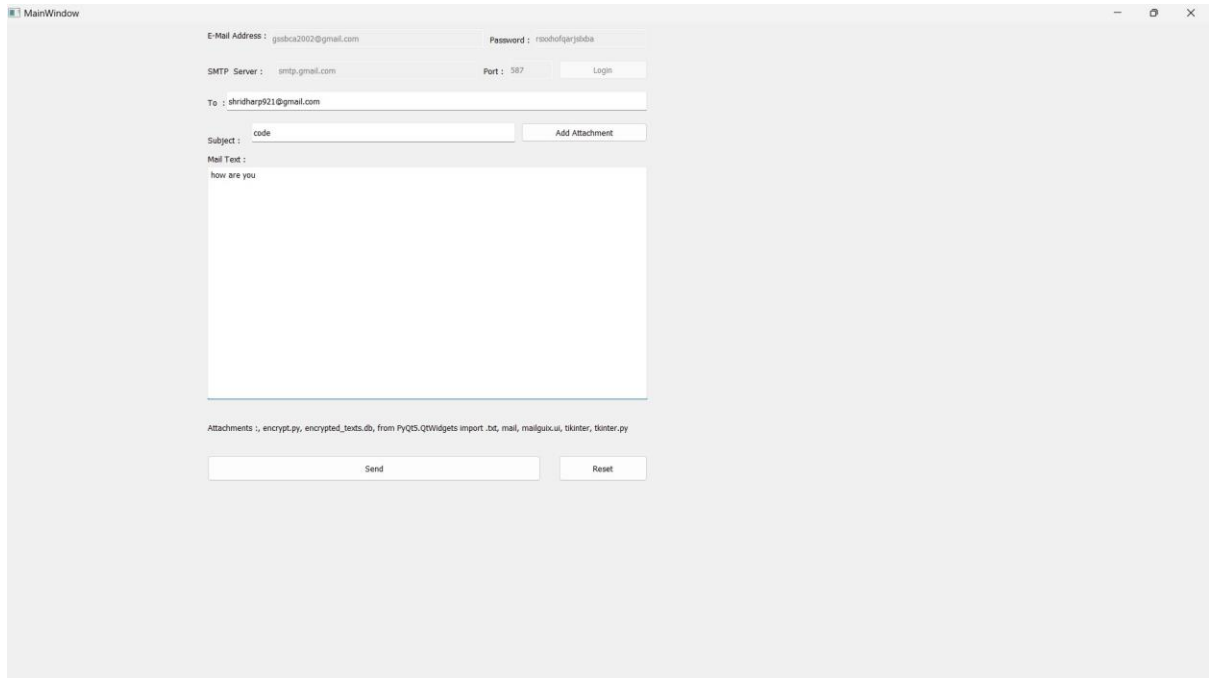
SECRET CODE

DATA FLOW DIAGRAM :



SECRET CODE

LAYOUTS :



SECRET CODE

SOURCE CODE :

```
from PyQt5.QtWidgets import *
from PyQt5 import uic
import smtplib
from email import encoders
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email.mime.multipart import MIMEMultipart

class MyGUI(QMainWindow):
    def __init__(self):
        super(MyGUI, self).__init__()
        uic.loadUi("mailguix.ui", self) # Assuming "mailgui.ui" is your UI file name
        self.show()

        self.pushButton.clicked.connect(self.login)
        self.pushButton_2.clicked.connect(self.attach_sth)
        self.pushButton_3.clicked.connect(self.send_mail)
        self.pushButton_4.clicked.connect(self.reset) # Connect "Reset" button
        self.server = None # Initialize the server variable

    def login(self):
        try:
            self.server = smtplib.SMTP(self.lineEdit_3.text(), int(self.lineEdit_4.text()))
            self.server.ehlo()
            self.server.starttls()
            self.server.ehlo()
            self.server.login(self.lineEdit.text(), self.lineEdit_2.text())

            self.lineEdit.setEnabled(False)
            self.lineEdit_2.setEnabled(False)
            self.lineEdit_3.setEnabled(False)
            self.lineEdit_4.setEnabled(False)
            self.pushButton.setEnabled(False)

            self.lineEdit_5.setEnabled(True)
            self.lineEdit_6.setEnabled(True)
            self.textEdit.setEnabled(True)
            self.pushButton_3.setEnabled(True)

            self.msg = MIMEMultipart()
        except smtplib.SMTPAuthenticationError:
            message_box = QMessageBox()
            message_box.setText("Invalid login Info!")
            message_box.exec_()
        except:
            message_box = QMessageBox()
```

SECRET CODE

```
message_box.setText("Login failed")

message_box.exec_()

def attach_sth(self):
    options = QFileDialog.Options()
    filenames, _ = QFileDialog.getOpenFileNames(self, "Open File", "", "All Files (*.*)",
options=options)
    if filenames:
        for filename in filenames:
            attachment = open(filename, 'rb')
            filename = filename[filename.rfind("/") + 1:]

            p = MIMEBase('application', 'octet-stream')
            p.set_payload(attachment.read())
            encoders.encode_base64(p)
            p.add_header("Content-Disposition", f"attachment; filename={filename}")
            self.msg.attach(p)
            self.label_8.setText(self.label_8.text() + ", " + filename)
            attachment.close()

def send_mail(self):
    dialog = QMessageBox()
    dialog.setText("Do you want to send this mail?")
    dialog.addButton(QPushButton("Yes"), QMessageBox.YesRole)
    dialog.addButton(QPushButton("No"), QMessageBox.NoRole)

    if dialog.exec_() == 0:
        try:
            self.msg['From'] = "NeuralNine"
            self.msg['To'] = self.lineEdit_5.text()
            self.msg['Subject'] = self.lineEdit_6.text()
            self.msg.attach(MIMEText(self.textEdit.toPlainText(), 'plain'))
            text = self.msg.as_string()
            self.server.sendmail(self.lineEdit.text(), self.lineEdit_5.text(), text)

            message_box = QMessageBox()
            message_box.setText("Mail Sent!")
            message_box.exec_()

        except:
            message_box = QMessageBox()
            message_box.setText("Sending Mail failed!")
            message_box.exec_()

def reset(self):
    # Reset the GUI elements and close the SMTP connection
    self.lineEdit.setEnabled(True)
    self.lineEdit_2.setEnabled(True)
    self.lineEdit_3.setEnabled(True)
```

SECRET CODE

```
self.lineEdit_4.setEnabled(True)

self.pushButton.setEnabled(True)

self.lineEdit_5.setEnabled(False)
self.lineEdit_6.setEnabled(False)
self.textEdit.setEnabled(False)
self.pushButton_3.setEnabled(False)
self.label_8.setText("")

if self.server:
    self.server.quit()
    self.server = None

app = QApplication([])
window = MyGUI()
app.exec_()
```

CONCLUSION :

In conclusion, the existing code provides the foundation for a simple email client application built using PyQt5 and smtplib. Here are the key points to summarize:

- **Project Overview:** The code creates a basic email client using PyQt5 and smtplib. Users can log in, compose/send emails, attach files, and reset the app.
- **Functionality:** Implements user authentication, email composition, and sending. Supports attaching files to emails.
- **User Interface:** Provides an easy-to-use graphical interface.
- **Feasibility:** Technically feasible using PyQt5 and smtplib. Economic and operational feasibility require further assessment.
- **Improvement Opportunities:** Enhance error handling and user feedback. Add advanced email features (e.g., inbox management, security). Optimize for better performance.
- **User Experience:** Suitable for basic email needs, but lacks advanced features.
- **Requirements and Dependencies:** Relies on Python, PyQt5, smtplib, and access to an SMTP server.

SECRET CODE

INSTALLATION :

To run the existing code, you'll need to install the required libraries and dependencies. Here are the steps to install and run the code:

- **Install Python:**
Ensure that you have Python installed on your system. You can download Python from the official website: <https://www.python.org/downloads/>
- **Install PyQt5:**
You can install PyQt5 using pip, the Python package manager. Open your command prompt or terminal and run the following command:
- **: pip install PyQt5**
- **No Additional Installation for smtplib:**
The smtplib library is part of Python's standard library, so you don't need to install it separately.
- **Download the Existing Code:**
Download the existing code and save it to a directory on your computer.
- **Run the Code:**
 - Open a command prompt or terminal, navigate to the directory where you saved the code, and run the Python script:
- **: python your_script_name.py**
Replace your_script_name.py with the actual name of the Python script that contains the code.
- **Use the Application:**
The PyQt5-based email client application should open. You can interact with it as described in the code's functionality.
- Please note that you might need to have an SMTP server's login credentials and server address to successfully use the email client to send emails. Make sure to enter the correct information when prompted by the application.
- Additionally, the code assumes that you have a UI file named "mailguix.ui" in the same directory as the Python script. If the UI file is located elsewhere, you may need to modify the path when loading the UI using uic.loadUi.

SECRET CODE

SCREENSHOTS :

